

End-to-End Quadrotor Position Control via Proximal Policy Optimization with Direct Motor Actuation

Adrian Ferdinand Jotham, Muhammad Wicaksono*, Soo Young Shin**

Adrian.jotham@gmail.com · muhammadwicak97@kumoh.ac.kr* · wdragon@kumoh.ac.kr**

Dept. of IT Convergence Engineering, Kumoh National Institute of Technology

Reinforcement Learning

Proximal Policy Optimization

Direct Motor Control

Curriculum Learning

Introduction & Contributions

Why end-to-end direct-RPM control?

- Classical control: 4 independently tuned layers (planner → position → attitude → motor mixer), each requiring explicit dynamic models and brittle to unmodelled dynamics.
- RL collapses all layers into one neural network — raw sensor readings map directly to motor commands, eliminating per-layer tuning.
- Direct-RPM is the hardest variant: every control layer must be learned simultaneously from one reward signal, facing quadratic thrust ($F = K_T \cdot n^2$), motor lag ($\tau \approx 0.15$ s) and the full inertia tensor — reward design becomes the central challenge.
- PPO demands reward density: rollout data is discarded after each update, so a dense, well-shaped signal is essential.
- Curriculum learning: spawn randomisation — position offset, velocity, tilt — is scaled by a coefficient $c \in [0, 1]$ that ramps over the first 1.5 M steps while reward weights anneal in parallel.



1. Reward Design

- Coupled potential-based approach term gives a dense transit gradient toward the goal.
- Velocity-gated hover bonus rewards arrival without overshoot — both jointly necessary for convergence.



3. Robustness Gap

- Distance-stratified success rates show graceful degradation with target distance.
- Robustness gap narrows monotonically: 47.1 → 28.3 pp.



2. Training Progression

- Learning characterised from 3M → 40M environment steps.
- Two spawn regimes: clean-start and full curriculum randomisation.

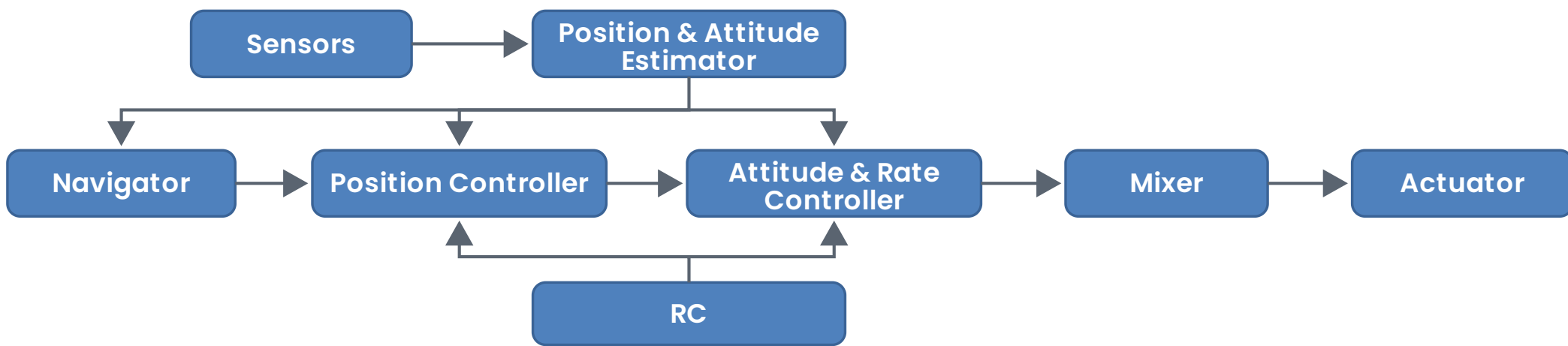


4. Open Environment

- Open Crazyflie 2.x training environment.
- Newton GPU rigid-body physics + Stable-Baselines3 PPO.

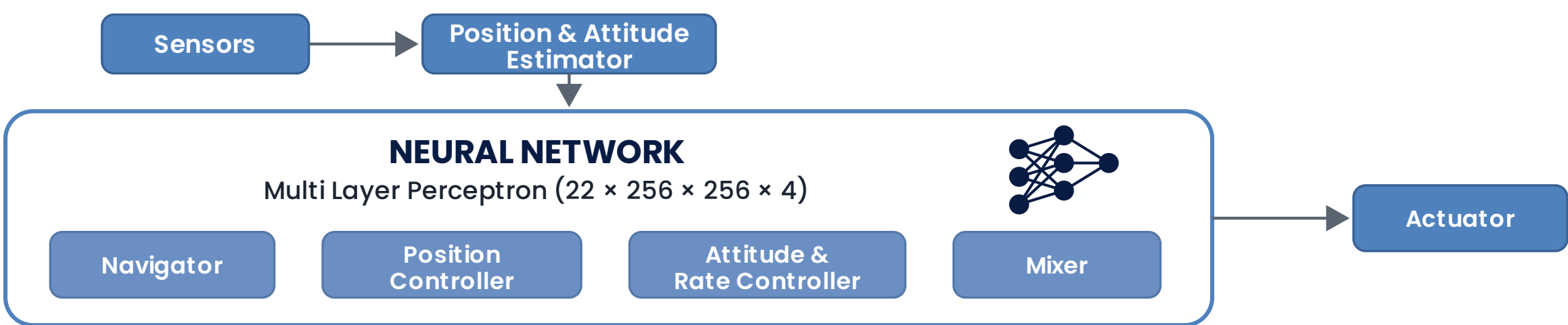
System Comparison

Classical Control Architecture



Src: PX4 Flight Stack

End to End Control Architecture



Method

- End-to-end: observation → PPO policy → motor RPM.
- Shared 2×256 MLP, separate actor + critic heads.
- Rollout (2,048 steps/env) exceeds the 800-step episode limit, so GAE returns are never truncated mid-episode.

Proximal Policy Optimization

Goal: $\pi^* = \arg \max_{\pi} \mathbb{E}_{\pi} [\sum_t \gamma^t r_t]$

Loss: $L^{\text{CIP}}(\theta) = \mathbb{E}_{\tau} [\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t)]$

Clipping bounds the policy step — the same rollout can be reused safely.

Observation & Action Space

22D observation: position error $p_{\text{err}} = \text{pos} - \text{target}$ (3D); rotation matrix R row-major (9D, avoids quaternion double-cover); linear velocity v (3D, world frame); angular velocity ω (3D, body frame); previous action a_{prev} (4D).

4D action maps normalised setpoints $\in [-1, 1]$ to motor RPM:

$$n_{\text{sp},i} = \text{clip}(14476 + a_i \cdot 7226, 1000, 21702) \text{ RPM}$$

Action 0 = hover equilibrium. A first-order LPF ($\tau = 0.15$ s) models motor lag.

Table 1. Crazyflie 2.x Physical Parameters

Parameter	Value
Mass	27 g
Arm length	32.5 mm
$I_{xx} = I_{yy}$	1.657×10^{-5} kg·m ²
I_{zz}	2.900×10^{-5} kg·m ²
K_T (thrust)	3.16×10^{-10} N / RPM ²
K_D (drag)	7.94×10^{-12} N·m / RPM ²
Hover RPM	$\approx 14,476$ RPM

System-identified Crazyflie 2.x parameters [3].

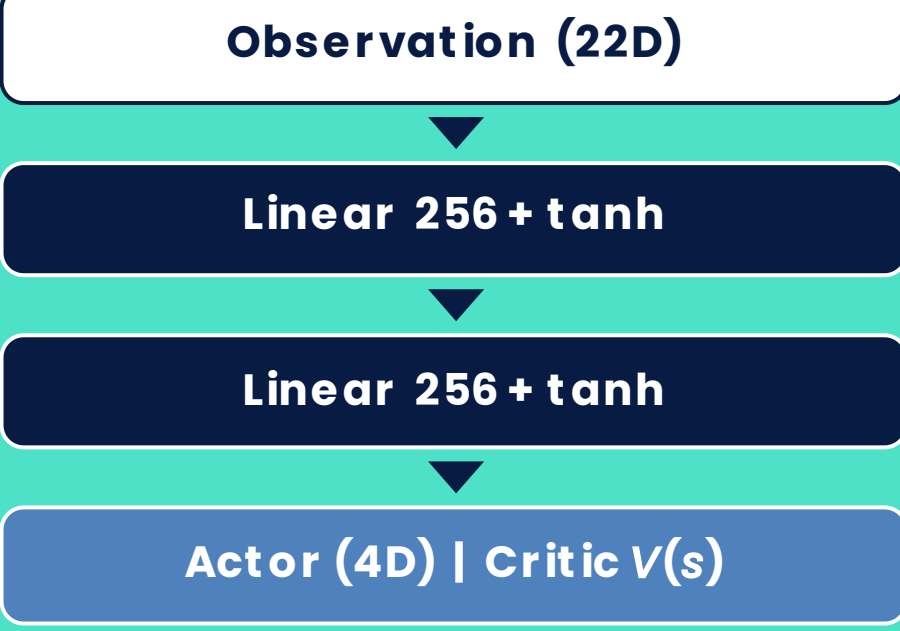


Simulation Environment

- 100 Hz integration via Newton GPU rigid-body engine (SolverSemImplicit).
- Motor lag $\tau = 0.15$ s (1st-order LPF).
- 16 parallel environments.
- Episodes ≤ 800 steps (8.0 s).



Network Architecture



PPO Configuration

- SB3 PPO [4], shared-trunk MLP 2×256 (tanh).
- Rollout 2,048 steps/env → 32,768 transitions/update.
- $n_{\text{epochs}} = 10$, batch = 64.
- $\gamma = 0.99$, $\lambda = 0.95$.
- clip = 0.2, ent_coef = 0.005.

Reward Function & Spawn Curriculum

Table 2. Reward Function Components

Term	Formula	Role
pos_c	$-C_{\text{ip}} \cdot \ p_{\text{err}}\ ^2$	Position precision
orient_c	$-C_{\text{ri}} \cdot (1 - q_w^2)$	Upright attitude
vel_c	$-C_{\text{iv}} \cdot \ v\ ^2$	Velocity damping
ang_c	$-C_{\text{ro}} \cdot \ \omega\ ^2$	Angular stability
act_c	$-C_{\text{ia}} \cdot \ \Delta a\ ^2$	Action jerk penalty
survival	+0.50 (fixed)	Stay airborne
approach	$+\lambda \cdot (d_{t-1} - d_t)$, $d > 0.10$ m	Dense transit signal
hover_bonus	$+0.15$, $d < 0.15$ m & $\ v\ < 0.5$ m/s	Settlement reward
arrival	+15.0 (one-time)	Waypoint reached
crash	-2.0 (one-time)	Ground impact

Weight ramp (first 1.5 M steps): $C_{\text{ip}} 0.05 \rightarrow 1.00$, $C_{\text{iv}} 0.01 \rightarrow 0.30$, $C_{\text{ro}} 0.001 \rightarrow 0.05$, $C_{\text{ia}} 0.005 \rightarrow 0.02$; $C_{\text{ri}} = 0.10$ fixed. The gated approach term suppresses oscillation incentives inside $d_{\text{gate}} = 0.10$ m while keeping a dense transit gradient up to the 0.15 m success boundary.

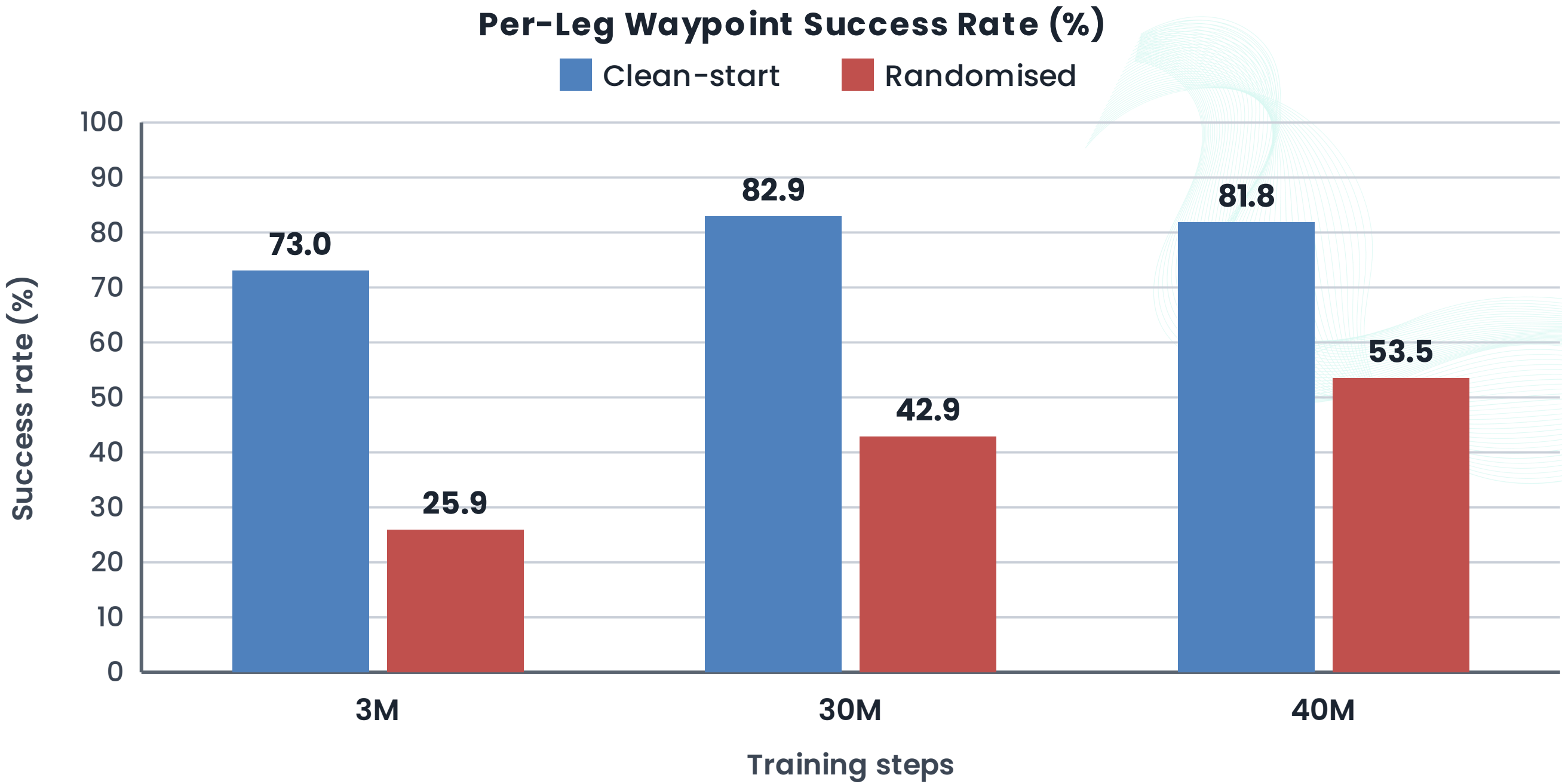
Why these terms

- Penalties (pos / orient / vel / ang / act) shape the long-tail target hovering behaviour.
- Approach term gives a dense gradient — pure goal-reach is too sparse for PPO.
- Velocity-gated hover bonus prevents the policy from sprinting at the target, which causes frequent overshoots.
- Arrival (+15) and crash (-2) provide the terminal signal.

Curriculum logic

- $c \in [0, 1]$ ramps spawn difficulty over the first 1.5 M steps; reward weights anneal in parallel — learn to hover first, then to recover.
- Spawn offsets: position $\leq (0.1 + 0.6c)$ m, velocity $\leq 1.5c$ m/s, angular rate $\leq 0.5c$ rad/s, tilt $\leq 15c^\circ$.
- $c = 0$ (clean start): stationary, upright, ≤ 0.1 m from waypoint. $c = 1.0$ (full randomisation): significant velocity and tilt requiring attitude recovery during navigation.

Results



How we measure it

- A leg = one waypoint segment. Success = $\|p_{\text{err}}\| < 0.15$ m.
- Multi-waypoint chaining (sequential waypoints, no reset), 8,000-step budget, 20 episodes. Clean-start ($c = 0$): stationary, upright spawn. Randomised ($c = 1.0$): full curriculum distribution.

Key findings

- Clean-start: 73.0 → 82.9% (3 → 30 M); converges at ~30 M (+9.9 pp, then -1.1 pp at 40 M).
- Randomised keeps improving through 40 M: 25.9 → 53.5%.
- Robustness gap narrows monotonically: 47.1 → 40.0 → 28.3 pp — robustness is a slowly-learned property that outlasts clean-start convergence.
- Distance-stratified: 100% at < 0.5 m degrades gracefully to 68.2% at 1.5–3.0 m.

Table 3. Learning Progression by Spawn Mode

Checkpoint	Clean-Start Success Rate	Clean-Start Waypoints / Ep.	Randomised Success Rate	Randomised Waypoints / Ep.	Robustness Gap
3M	73.0%	2.7 ± 2.3	25.9%	0.3 ± 0.7	47.1 pp
30M	82.9%	4.8 ± 3.2	42.9%	0.8 ± 0.8	40.0 pp
40M	81.8%	4.5 ± 3.6	53.5%	1.1 ± 1.8	28.3 pp

Conclusion & Future Work

- End-to-end PPO learned direct-RPM position control — no hand-tuned cascade.
- Training to 40 M steps achieves 81.8% per-leg waypoint success under clean-start conditions (convergence at ~30 M with 82.9%).
- Robustness keeps improving past clean-start convergence — randomised performance rises 25.9 → 53.5% while the gap narrows 47.1 → 28.3 pp.



Sim-to-Real

- Validate the simulator: zero-shot transfer to a physical Crazyflie 2.x.
- Real motor lag and battery sag will stress the policy.



Motor-Lag Robustness

- Randomise τ around 0.15 s during training (domain randomisation).
- Closes the gap between sim motor dynamics and any unseen real airframe.



Reward Refinement

- Remove flat +0.5 survival bonus — currently creates a hover attractor away from the goal.
- Replace with a small altitude/recovery term.

REFERENCES

- [1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv preprint arXiv:1707.06347, 2017.
- [3] D. Förster, "System Identification of the Crazyflie 2.0," B.Sc. thesis, ETH Zürich, 2015.

- [2] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum learning," in Proc. 26th Annual Int. Conf. on Machine Learning (ICML), 2009, pp. 41–48.

- [4] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable reinforcement learning implementations," J. Machine Learning Research, vol. 22, no. 268, pp. 1–8, 2021.

